Tools and the Road to MCP

From Prompt Engineering to Agent Infrastructure

Bryan Bischof

18 Feb 2026

Today's Journey

Last week: prompt engineering—the model's text input as the primary lever.

This week: **what happens when text isn't enough?**

The Through-Line

Language models want to act in the world, and we've been inventing better ways to let them. Each milestone solves a real limitation of the previous one.

Milestones 1–4:

1. Research roots (pre-2021)
2. LLMs reach for tools (2021–22)
3. Training tool use (2023)
4. Platform features (2023–24)

Milestones 5–8:

5. MCP (Nov 2024)
6. Agent-to-Agent / A2A (2025)
7. Subagents (2025)
8. Skills (Oct 2025)

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Where Prompt Engineering Hits a Wall

In Unit 4 you learned powerful prompt patterns. But consider three requests:

1. "What's the current price of AAPL stock?"
2. "Book me a meeting with Priya at 2pm tomorrow."
3. "Run my test suite and fix any failures."

No amount of prompt crafting helps. The model doesn't have live data, can't take actions with effects, and can't execute code. It can only *produce text*.

Is there a way to make generated text actionable in systems?

That question—and the increasingly sophisticated answers to it—is the subject of today's lecture.

The Tool-Use Loop (Key Mental Model)

Every milestone we cover is a variation on this loop:

1. User sends a request
2. Model decides: "I need to call tool X with arguments Y"
3. *Something* executes that call (your code, the provider, an MCP server...)
4. The result comes back to the model
5. Model continues (possibly with more tool calls)

At every milestone, we answer the same **three questions**:

1. **How does the model express the call?** (step 2)
2. **Who or what executes it?** (step 3)
3. **What structural guarantees do you get?** (deterministic, enforced, or aspirational?)

Each answer becomes a row in the **comparison matrix** we're building together.

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Why It Matters

Long before LLMs, NLP researchers built systems that translate natural language into executable commands: SQL queries, API calls, robot instructions.

Two key traditions:

- **Semantic parsing:** map a user's sentence into a structured program that a runtime can execute.
- **Task-oriented dialogue:** multi-turn pipeline—track dialogue state, query a database/API, generate a response.

Modern tool use compresses these pipelines, but the architecture is the same.

Semantic Parsing: Example

A natural-language interface to a weather API:

```
User: "What's the forecast for Chicago tomorrow?"
      ↓ parser
GET /forecast?location=chicago&date=2026-02-19
      ↓ runtime executes HTTP request
{"high": 28, "low": 15, "conditions": "snow"}
      ↓ response generator
"Tomorrow in Chicago: high of 28°F, low of 15°F, snow expected."
```

What's inside the parser? Grammar rules or pattern templates, written by hand. Works perfectly for the patterns you wrote—fails silently on everything else.

Want to handle "weather next Tuesday in Chicago"? Write another rule. Want open-ended questions? You can't—this approach doesn't scale.

Semantic Parsing: Parser Code

Simplified rule-based parser

```
import re
from datetime import date, timedelta

PATTERNS = [
    (r"forecast for (\w+) tomorrow",
     lambda m: {"endpoint": "/forecast",
                "params": {"location": m.group(1).lower(),
                           "date": (date.today() + timedelta(days=1)).isoformat()}}),
    (r"weather in (\w+)",
     lambda m: {"endpoint": "/current",
                "params": {"location": m.group(1).lower()}}),
]

def parse(utterance: str) -> dict | None:
    for pattern, builder in PATTERNS:
        m = re.search(pattern, utterance, re.IGNORECASE)
        if m:
            return builder(m)
    return None  # no matching rule
```

Task-Oriented Dialogue: Example

A flight-booking dialogue system (think Siri-era):

```
User: "I need a flight to Denver next Friday."
      ↓ dialogue state tracker
{intent: book_flight, dest: "DEN", date: "2026-02-27", origin: null}
      ↓ system realizes origin is missing
Agent: "Where are you flying from?"
User: "Newark."
      ↓ state updated: {origin: "EWR", ...}
      ↓ database query
SELECT * FROM flights
  WHERE origin='EWR' AND dest='DEN' AND date='2026-02-27';
      ↓ results formatted into response
Agent: "I found a United flight departing 8:15am for $247..."
```

Each component—intent classifier, slot filler, state tracker, query builder, response generator—is a separate module. The "LLM tool use" story is about collapsing most of these into a single model.

Task-Oriented Dialogue: State Tracker Code

Slot-filling state tracker

```
from dataclasses import dataclass

SLOT_SCHEMA = {
    "origin": {"type": "airport_code", "required": True},
    "dest":   {"type": "airport_code", "required": True},
    "date":   {"type": "date",         "required": True},
}
PROMPTS = {
    "origin": "Where are you flying from?",
    "dest":   "Where are you flying to?",
    "date":   "What date do you want to fly?",
}

@dataclass
class DialogueState:
    intent: str | None = None
    slots: dict[str, str | None] = None

    def __post_init__(self) -> None:
        if self.slots is None:
            self.slots = {k: None for k in SLOT_SCHEMA}

    def next_action(self) -> str:
        for slot, meta in SLOT_SCHEMA.items():
            if meta["required"] and self.slots.get(slot) is None:
                return PROMPTS[slot]
        return "ready"
```

Milestone 1: Key Insight & Loop Check

Key Insight

A language model is a conditional translator: natural language in, structured command out. Execution happens elsewhere.

This idea never goes away—it's the through-line to MCP.

Loop check — semantic parsing

1. *How does the model express the call?* A specialized parser maps NL to a structured command. No LLM in the loop yet.
2. *Who executes?* An external runtime (database engine, API server).
3. *What guarantees?* Deterministic within the parser's grammar—if it parses at all, the output is well-formed. But coverage is narrow.

Approach	Who decides?	Who executes?	Output format
Semantic parsing	Specialized parser	External runtime	SQL / API call

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

The Capability Explosion

Once large pretrained models became strong instruction-followers, people realized they could *prompt* them to emit tool calls.

Several landmark papers in just two years:

- **WebGPT** (2021) — browse as a tool
- **MRKL** (2022) — the modular blueprint
- **PAL** (2022) — write code, execute it
- **SayCan** (2022) — select from available skills
- **ReAct** (2022) — reasoning + acting, interleaved

Each one expanded the **capability envelope** of what LLMs could do. Before this era, LLMs were chatbots. After it, they were starting to become labor.

WebGPT — Browse as a Tool (2021)

OpenAI trained a model to interact with a browser environment (search, click, quote, scroll) and optimized with human feedback.

New capabilities: factual QA with citations over live web content. The model could answer questions about events after its training cutoff.

What the model outputs: structured browser commands in a custom text format—`Search`, `Click`, `Quote`, `Scroll`—interpreted by a browser environment.

How the model learned this: *not* free from prompting. OpenAI collected human demonstrations of browsing behavior, trained via behavior cloning, then refined with RLHF.

Read

Nakano et al., *WebGPT: Browser-Assisted Question-Answering with Human Feedback*, 2021.
`arxiv.org/abs/2112.09332`

MRKL — The Modular Blueprint (2022)

MRKL (Modular Reasoning, Knowledge and Language): combine a general LM with specialized modules—calculator, database, search engine—each accessible as a "tool."

New capabilities: accurate arithmetic, database queries, real-time lookups. The LLM handles language; the modules handle precision.

What the model outputs: a routing decision ("this question needs the calculator module") plus a text input for that module.

How the model learned this: MRKL proposed training a dedicated router—a classifier that maps user inputs to the right expert module. *Not* something you get from few-shot prompting.

Read

Karpas et al., *MRKL Systems*, 2022. `arxiv.org/abs/2205.00445`

PAL — Write Code, Execute It (2022)

Program-Aided Language Models: instead of forcing the model to do arithmetic in text, have it write Python, execute the code, and use the output.

New capabilities: reliable math, symbolic reasoning, data transformations—anything you can express in code.

PAL output is executable code

```
def solve():
    # Problem: tax on $47.50 at 8%
    price = 47.50
    tax_rate = 0.08
    total = price * (1 + tax_rate)
    return total
# Runtime executes solve() -> returns 51.30
```

How the model learned this: one of the rare wins largely free from prompting. Uses few-shot prompting on code-trained LMs (Codex). The base model's code-training already encoded "how to translate word problems into programs."

Read: Gao et al., *PAL: Program-Aided Language Models*, 2022. `arxiv.org/abs/2211.10435`

SayCan — Selecting from Available Skills (2022)

SayCan combined an LM's semantic scoring with a robot's feasibility constraints: the LM says what *should* happen, the affordance model says what *can* happen.

New capabilities: grounded physical action. A language model's "understanding" of tasks translates into a real robot doing things in the world.

What the model outputs: the LM scores candidate actions from a pre-defined skill menu (e.g. "pick up the sponge", "go to the counter") and the system selects the highest-scoring *feasible* action. Output is a probability distribution over a closed action set, not free text.

How the model learned this: the LM's scoring comes from pre-training. The affordance functions were trained separately via RL on the robot. The *architecture* is the contribution.

Read: Ahn et al., *Do As I Can, Not As I Say*, 2022. `arxiv.org/abs/2204.01691`

ReAct — Reasoning + Acting, Interleaved (2022)

ReAct formalized the prompting pattern that dominates agent frameworks today: interleave *reasoning traces* with *actions* and *observations*.

New capabilities: multi-step research tasks requiring planning.

ReAct trace

```
Thought: I need to find who directed Jaws.
Action: search("Jaws 1975 director")
Observation: Jaws was directed by Steven Spielberg.
Thought: Now I need to check if Spielberg directed a dinosaur movie.
Action: search("Steven Spielberg dinosaur movie")
Observation: Spielberg directed Jurassic Park (1993).
Thought: Yes --- Spielberg directed both.
Action: finish("Yes, Steven Spielberg directed both.")
```

How the model learned this: started as a prompting technique, but then **providers trained on these traces.** Models trained on reasoning traces follow the format more reliably and recover from errors better.

Read: Yao et al., *ReAct*, ICLR 2023. `arxiv.org/abs/2210.03629`

Aside: Reasoning Models and Test-Time Compute

The ReAct pattern has a deeper descendant: **reasoning models** (o1, o3, Claude with extended thinking, DeepSeek-R1).

Two advances beyond ReAct:

1. **Training on chains of thought.** Massive datasets of step-by-step reasoning traces (sometimes self-generated and filtered—STaR).
2. **Test-time compute scaling.** More inference-time compute via longer chains, exploring multiple paths, re-checking work. Trade compute for accuracy.

Why this matters for tools:

- Reasoning models make better tool-use decisions
- "I should search for X, but if that fails try Y" — fewer wasted tool calls
- Step-change improvements in agentic benchmarks (SWE-bench)

The Recurring Pattern

A prompting innovation shows what's possible → training turns it from fragile to robust → test-time compute scaling makes it better again.

The Capability Envelope So Far

Paper	What LLMs couldn't do before	What they can do now
WebGPT	Answer Qs about current events	Browse the web, cite sources
MRKL	Do precise math, query databases	Route to specialized modules
PAL	Solve multi-step math reliably	Write + execute code
SayCan	Act in the physical world	Select feasible robot actions
ReAct	Plan multi-step research tasks	Chain reasoning with tool calls

LLMs went from "impressive chatbot" to "system that can browse, compute, query, and act."

The Training Story

Most of these were **not free.** WebGPT needed RLHF. MRKL needed a trained router. SayCan needed RL-trained affordance functions. Only PAL and ReAct worked primarily through prompting—and both leaned on code pre-training. When you see a new capability, ask: "what training made this possible?"

Milestone 2: Loop Checks

Loop check — prompted tool use (ReAct, MRKL)

1. *How does the model express the call?* Free-form text following a convention you defined in the prompt (e.g. `Action: search(...)`).
2. *Who executes?* Your glue code—regex parsing, dispatching to APIs.
3. *What guarantees?* None. The model *might* follow your format, or it might drift, hallucinate tool names, or produce unparseable arguments. You're parsing on hope.

Hallucinated tool names and malformed arguments are failure modes (Unit 3) that deserve their own invariants in your eval set.

Loop check — code generation (PAL)

1. *How does the model express the call?* It writes executable code.
2. *Who executes?* A Python interpreter (or similar runtime).
3. *What guarantees?* Partial. The interpreter enforces syntax, but semantics are unconstrained—the code can still be wrong, unsafe, or call nonexistent APIs.

Comparison Matrix Update

Approach	Who decides?	Who executes?	Output format
Semantic parsing	Specialized parser	External runtime	SQL / API call
Prompted tool use	**LLM via prompt**	**Your glue code**	**Free-form text / XML**
Code generation (PAL)	**LLM writes code**	**Python interpreter**	**Executable code**

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

From Prompting to Training

Milestone 2 showed LLMs can use tools, but the training story was varied. What none of them did was teach a model *general-purpose* tool use—when to call which tool, with what arguments, across an open-ended set of APIs.

Two landmark efforts:

- **Toolformer:** self-supervised tool learning on a handful of tools
- **Gorilla:** training for massive, changing API surfaces

Toolformer — Self-Supervised Tool Learning

A model can teach *itself* when to call tools:

1. Start with plain training text (e.g. "The population of Toronto is 2,794,356")
2. For each position, ask: would inserting a tool call here have helped predict the next tokens better?
3. Try candidate calls and keep ones that reduce perplexity
4. Fine-tune on the augmented text

Tool calls spliced into training data

```
The population of Toronto is [QA("population of Toronto")] -> 2,794,356
The square root of 256 is [Calculator(sqrt(256))] -> 16.0
```

Key difference from prompting: the format is in the weights, not the prompt.

Limitation: closed, pre-defined tool set (5 tools). Each new tool requires re-running the pipeline. This is the gap Gorilla targets.

Read: Schick et al., *Toolformer*, NeurIPS 2023. `arxiv.org/abs/2302.04761`

Gorilla — Training for Massive API Surfaces

Gorilla targeted the real-world scaling problem: thousands of APIs, complex signatures, changing docs.

Training data: (instruction, API call) pairs

```
Instruction: "I want to classify images using a pre-trained
              model that works well on ImageNet"
API call:    hub.load('pytorch/vision', 'resnet152', pretrained=True)
```

Fine-tuned on thousands of pairs across HuggingFace, TorchHub, and TensorHub. Uses **Retriever-Aware Training (RAT)**: API documentation retrieved and included in context during both training and inference.

New capabilities: correct API calls across massive, changing surfaces. Also introduced **APIBench** for evaluation.

Why this matters for what comes next

Gorilla's approach—fine-tuning on (task, API call) pairs + retrieval over API docs—is directly ancestral to how modern providers handle tool use.

Read: Patil et al., *Gorilla*, NeurIPS 2024. `arxiv.org/abs/2305.15334`

Milestone 3: Loop Check & Matrix Update

Loop check — trained tool use (Toolformer, Gorilla)

1. *How does the model express the call?* A learned call format baked into the weights—the model was trained on successful tool-call examples.
2. *Who executes?* Still your glue code, but the calls are better-formed.
3. *What guarantees?* Statistical, not structural. Training makes correct format *likely* but nothing enforces it at decode time. You still need validation.

Approach	Who decides?	Who executes?	Output format
Semantic parsing	Specialized parser	External runtime	SQL / API call
Prompted tool use (ReAct)	LLM via prompt	Your glue code	Free-form text / XML
Code generation (PAL)	LLM writes code	Python interpreter	Executable code
Trained tool use	**LLM (fine-tuned)**	**Your glue code**	**Learned call format**

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

The Principle

This is where tool use leaves the research lab and becomes something you configure in an API call.

Don't over-engineer

If you only need structured data, don't jump to tools. A prompt that asks for JSON output + a Pydantic validator is simpler, cheaper, and faster than wiring up tool calling. Tools are for when the model needs to *do* something—fetch data, trigger a side effect, run code.

The XML-Tag Era — Prompt-Invented Protocols

Before providers shipped official tool-calling APIs, developers invented conventions in prompts:

Prompt convention (early Claude + LangChain)

```
If you need a tool, respond EXACTLY in this format:
  Tool: weather_lookup
  Input: {"city": "New York"}
Then STOP and wait for the result.
```

The model would respond:

```
I need to check the weather.
Tool: weather_lookup
Input: {"city": "New York"}
```

Everything that can go wrong: model might not include the `Tool:` prefix, emit malformed JSON, hallucinate tool names, or put extra text that breaks your regex. Every failure mode is a string-parsing bug.

XML-Tag Era: Dispatch Code

Your runtime parses and dispatches

```
import re, json
from typing import Any

TOOL_PATTERN = re.compile(
    r"Tool:\s*(\w+)\s*\nInput:\s*(\{.*\})", re.DOTALL,
)

def dispatch_tool(name: str, args: dict[str, Any]) -> str:
    if name == "weather_lookup":
        return get_weather(**args)
    if name == "stock_price":
        return get_stock_price(**args)
    raise ValueError(f"Unknown tool: {name}")

def maybe_handle_tool_call(model_response: str) -> str | None:
    match = TOOL_PATTERN.search(model_response)
    if not match:
        return None
    tool_name = match.group(1)
    tool_args = json.loads(match.group(2))
    return dispatch_tool(tool_name, tool_args)
```

This is not "true" tool use by itself—it's a protocol *you* invented in prompts. The fragility of this parsing code is exactly why providers eventually built structured tool-calling into their APIs.

Aside: The Road Toward Agents

Notice what the XML-tag pattern is *almost* doing: observe → decide → act → observe → repeat. That's the skeleton of an **agent**.

But in this era, none of the pieces are truly agentic yet:

- **No loop:** the developer's code handles control flow. The model doesn't decide "should I call another tool or am I done?"
- **No planning:** the model responds to one turn at a time.
- **No autonomy:** every tool call requires your parsing code; every result requires you to inject it back manually.

The *interface* for agents is being invented—the idea that models can emit structured actions and receive observations—even though the *systems* around them aren't autonomous yet. We'll cover full agentic patterns in Unit 9.

OpenAI Plugins — Tools via OpenAPI (Mar 2023)

ChatGPT Plugins: tools defined by a web API, described with `ai-plugin.json` + OpenAPI schema, which the model could discover and call.

Under the hood: OpenAI took each plugin's manifest and OpenAPI spec and **injected them into the system prompt**. The model was fine-tuned to read these specs and emit correct API calls.

Security concerns: researchers showed malicious plugins could inject instructions via API responses (prompt injection through tool output), and the model could be tricked into chaining plugins to exfiltrate user data.

This normalized key ideas:

- Tools are **externally described** (schema, not prompt hacking)
- Models can **select and call them**
- Tool descriptions matter for both **safety and correctness**
- Tool-use capability comes from fine-tuning on schema/call pairs

Read: OpenAI Plugins blog post (since deprecated in favor of GPT Actions)

Function Calling — The Inflection Point (Jun 2023)

Arguably the single biggest inflection point in practical tool use.

Everything before—XML tags, regex parsing, prompt-invented protocols—required you to *hope* the model would emit the right format. Function calling changed the contract:

The API itself becomes the structured channel

You declare tools as JSON Schema definitions in the API request. The model returns structured arguments in a dedicated field—not embedded in prose, not wrapped in XML, but as a first-class part of the API response.

Before function calling:

- Regex parsing
- Brittle glue code
- One malformed response = crash

After function calling:

- Clean, typed arguments
- No regex, no hope
- Production-grade primitive

Function Calling: Code Example

Declaring a function and getting structured output

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"},
                "units": {"type": "string",
                          "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["city"]
        }
    }
}]

response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Weather in NYC?"}],
    tools=tools,
)
# response.choices[0].message.tool_calls contains:
#   function.name = "get_weather"
#   function.arguments = '{"city": "New York", "units": "fahrenheit"}'
```

Read: OpenAI Function Calling guide: `platform.openai.com/docs/guides/function-calling`

JSON Mode — Valid JSON Without Schema Guarantees (Nov 2023)

Set `response_format: {"type": "json_object"}` and the model is constrained to produce valid JSON.

What it actually guaranteed: `json.loads()` wouldn't crash. That's it.

What went wrong in practice:

- Omitted required fields unpredictably
- Invented field names not in the requested schema
- Returned objects when you asked for arrays (and vice versa)
- Occasionally produced incomplete JSON at the token limit

This led to retry-and-validate wrappers: call the API, try `json.loads()`, check with Pydantic, retry on failure. Libraries like **Instructor** and **Marvin** were built to paper over this gap.

When this helps: you just need parseable JSON and can validate structure yourself.

When it's not enough: you need guaranteed field names, types, or enums.

Structured Outputs — Schema-Enforced (Aug 2024)

OpenAI introduced Structured Outputs to ensure model responses conform *exactly* to a developer-supplied JSON Schema. This uses **constrained decoding**—the model literally cannot produce tokens that would violate the schema.

How constrained decoding works

At each step, the system computes a **token mask**: which tokens are valid given the current position in the schema? Every invalid token gets its probability set to zero.

If the schema says the next field must be `"city"` and the model has emitted `{"ci`, the mask allows `ty"` and disallows everything else.

```
Normal:     P(token) → sample → next token
Constrained:  P(token) → mask invalid → sample → next token
```

OpenAI's implementation (LLGuidance) converts JSON Schemas into CFGs, computes masks in $\sim$50 μs per token.

The Determinism Ladder

Level	Guarantee
Free text	No guarantees, you parse and hope
XML-tag prompting	No guarantees, you regex and hope harder
JSON mode	Valid JSON guaranteed, schema not enforced
Structured outputs	Schema-conformant, enforced at decode time

Constrained decoding makes schema conformance a property of the *generation process*, not a post-hoc validation check. It's the difference between "the model usually gets it right" and "the model structurally cannot get it wrong."

Each rung also expands the set of invariants (Unit 3) you can write—schema-enforced outputs let you validate every field deterministically.

Read: OpenAI Structured Outputs: `openai.com/index/introducing-structured-outputs-in-the-api/`

Anthropic Tool Use — `tool_use` / `tool_result` Blocks

Same conceptual loop as OpenAI's function calling, but different wire format.

Anthropic: tool calls are **content blocks**—inline in the same `content` array as text:

```
{"role": "assistant",
 "content": [
   {"type": "text",
    "text": "Let me check..."},
   {"type": "tool_use",
    "id": "call_01",
    "name": "get_weather",
    "input": {"city": "New York"}}
 ]}
```

OpenAI: tool calls in a dedicated field (`message.tool_calls`), separate from text content.

Anthropic's content-block model makes it natural for the model to "narrate" its tool use. OpenAI's separation is cleaner for parsing.

Neither is strictly better; your dispatch code will look different.

What's the same: JSON Schema tool definitions, structured arguments, typed response channel. No more regex parsing.

Read: Anthropic Tool Use docs: `docs.anthropic.com/en/docs/build-with-claude/tool-use/overview`

Milestone 4: Loop Check

Loop check — function calling / structured outputs

1. *How does the model express the call?* Via a provider API—the model emits structured JSON arguments guided by a schema you supply.
2. *Who executes?* Your code. You receive the structured arguments and dispatch the actual function call.
3. *What guarantees?* This is where determinism actually arrives. Function calling gives you valid JSON with schema guidance. Structured outputs use constrained decoding—the model *cannot* produce tokens that violate the schema. Format conformance goes from "usually" to "always."

Comparison Matrix Update

Approach	Who decides?	Who executes?	Output format	Format guarantees
Semantic parsing	Specialized parser	External runtime	SQL / API call	Deterministic (within grammar)
Prompted tool use	LLM via prompt	Your glue code	Free-form text	None (hope-based)
Code generation (PAL)	LLM writes code	Interpreter	Executable code	Syntax only
Trained tool use	LLM (fine-tuned)	Your glue code	Learned format	Statistical
Function calling	**LLM via API**	**Your code**	**JSON (schema-guided)**	**Valid JSON**
Structured outputs	**LLM via API**	**Your code**	**JSON (schema-enforced)**	**Schema-deterministic**

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

The Problem MCP Solves

Function calling solved: *"How do I get the model to emit a well-structured call in my app?"*

But it created a new scaling problem:

5 tools, 1 model?
Function calling is fine.

50 tools across GitHub, Slack, Postgres, Stripe, Jira—available to Claude Desktop, Cursor, your custom agent, a teammate's notebook?
You're writing $N \times M$ bespoke connectors.

MCP solves

"How do we stop rewriting connectors for every model/app/tool combination?"

What MCP Is

Anthropic announced and open-sourced the **Model Context Protocol** on November 25, 2024. Inspired by the Language Server Protocol (LSP).

Architecture:

```
Host (Claude Desktop, Cursor, your app)
  ↪ Client (speaks MCP)
      ↪ Server (exposes tools, resources, prompts via JSON-RPC 2.0)
          ↪ Your system (DB, API, filesystem, etc.)
```

An MCP server exposes three primitives:

- **Tools:** executable functions the model can call
- **Resources:** structured data the model can read
- **Prompts:** templated instructions for common workflows

Clients discover what's available via `tools/list`, `resources/list`, etc.

Governance: Anthropic donated MCP to the Linux Foundation's Agentic AI Foundation in December 2025—vendor-neutral standard.

When to Use MCP vs. Plain Function Calling

Prefer function calling when:

- Small, stable set of internal functions
- Maximal control inside one backend
- Tools don't need to be reused across multiple hosts/apps

Prefer MCP when:

- Tools live across many systems (GitHub + Slack + DB + payments + ...)
- Standardized server that multiple clients can connect to
- Need tool discovery (`tools/list`) and centralized policy

Two underappreciated properties:

Distribution

An MCP server is a standalone process—different machine, container, behind a firewall. The team that owns a system builds and ships the MCP server; every agent host connects. Publish once, everyone connects.

MCP: Data Modeling via Resources

Function calling only gives you tools—callable functions. MCP adds **resources**: structured, read-only data that the model can browse and pull into context.

This separates two concerns that function calling lumps together:

- **Tools** answer "what can I do?" (actions with side effects)
- **Resources** answer "what can I see?" (structured data to reason over)

An agent that can browse resources before deciding which tool to call makes better decisions—it has context.

Example

A database MCP server doesn't just expose a `query` tool; it exposes tables as resources that clients can list, preview, and selectively load.

MCP in Practice: Code Example

A minimal MCP server in Python (using the `mcp` SDK):

```
from mcp.server import Server
from mcp.types import Tool, TextContent

app = Server("weather-server")

@app.tool()
async def get_weather(city: str) -> list[TextContent]:
    """Get current weather for a city."""
    data = await fetch_weather(city)
    return [TextContent(
        type="text",
        text=f"{city}: {data['temp']}F, {data['conditions']}"
    )]
```

Any MCP-compatible host (Claude Desktop, Cursor, a custom agent) can now discover and call `get_weather` without knowing anything about its implementation.

MCP vs. Other Approaches (Common Confusions)

MCP vs. Plugins/Apps:
OpenAI's plugins are a distribution + UX concept tied to one platform. MCP is a vendor-agnostic protocol.

MCP vs. RAG:
RAG retrieves text. MCP provides retrieval (resources) *plus* actions (tools) with structured I/O and authorization.

Practical tips (hard-earned):

- Treat MCP servers like real APIs: auth, audit logs, rate limits, versioning
- Avoid tool explosion—models degrade when the tool list gets long
- Use approval policies for dangerous calls (payments, deletions, production writes)

Milestone 5: Loop Check & Matrix Update

Loop check — MCP

1. *How does the model express the call?* Same as function calling (the host translates MCP tool schemas into the model's native tool format).
2. *Who executes?* An MCP server—a separate process that owns the tool implementation. Your host just proxies.
3. *What guarantees?* Inherits function-calling determinism for the call format. MCP adds a new guarantee: *discoverability*—clients enumerate available tools and schemas at runtime via `tools/list`.

Read: Anthropic MCP announcement, MCP spec (`modelcontextprotocol.io`), OpenAI MCP tool guide

Approach	Who decides?	Who executes?	Format	Guarantees	Reusability
Semantic parsing	Spec. parser	External runtime	SQL / API call	Deterministic	Low
Prompted tool use	LLM via prompt	Your glue code	Free-form text	None	Low
Code generation	LLM writes code	Interpreter	Executable code	Syntax only	Low
Trained tool use	LLM (fine-tuned)	Your glue code	Learned format	Statistical	Low
Function calling	LLM via API	Your code	JSON (schema)	Valid JSON	Medium
Structured outputs	LLM via API	Your code	JSON (enforced)	Schema-det.	Medium
MCP	**LLM via API**	**MCP server**	**JSON-RPC**	**Schema-det. + disc.**	**High**

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Why Agents Talking to Agents Matters

MCP connects an agent to *tools and data*. But sometimes a tool isn't the right abstraction—you need *another agent*.

Why tools aren't always enough. A tool is a function: deterministic input, deterministic output, no judgment. But consider:

- "Negotiate a contract with this vendor" — back-and-forth, judgment, domain expertise
- "Review this PR for security issues" — reading code, reasoning about attack surfaces
- "Process this insurance claim" — touches underwriting, fraud detection, customer communication, payment

These don't fit the tool interface. You need something that can receive an open-ended task, work autonomously, and report back—possibly over minutes or hours.

The N×M Problem, Again

Without a standard, every agent-to-agent integration is bespoke—the same problem MCP solved for tools, but at the agent level.

A2A: The Protocol

Google announced the Agent2Agent Protocol (A2A) in April 2025 as an open protocol for agent interoperability—explicitly complementing MCP:

MCP:
"Let my agent use external tools/data sources."

A2A:
"Let my agent delegate to / collaborate with other agents."

Key difference from a tool call

A2A tasks are **opaque and long-lived**. The requesting agent doesn't know (or care) how the remote agent accomplishes the task. It might involve multiple tool calls, human approvals, sub-delegations. The protocol handles capability discovery, task lifecycle management, and secure communication. Built on HTTP, SSE, and JSON-RPC.

Read: A2A announcement (`developers.googleblog.com`), A2A spec (`google.github.io/A2A/specification/`)

Milestone 6: Loop Check

Loop check — A2A

1. *How does the model express the call?* The agent discovers a peer's capabilities and sends a structured task request via the protocol.
2. *Who executes?* A remote agent—potentially on a different team, org, or vendor's infrastructure.
3. *What guarantees?* The protocol layer is deterministic (structured messages, capability negotiation). But the remote agent's *behavior* is opaque—you get protocol guarantees, not outcome guarantees. Trust boundaries matter.

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

The Problem Subagents Solve

Important distinction

Subagents are an **architecture pattern**, not a protocol. MCP and A2A are wire protocols. Subagents are a design decision *within* your system: how you decompose work internally.

Long-running agent sessions hit three chronic issues:

1. **Context overflow:** exploration fills the window with noise
2. **Permission sprawl:** a research subtask shouldn't have write access to production databases
3. **Specialization:** different subtasks benefit from different system prompts, tools, and even models

How Subagents Work

A subagent is a child agent with its own context window, custom system prompt, restricted/expanded tool set, and optionally a different (cheaper/faster) model.

Cursor's subagent pattern

```
Parent agent (main task: "refactor the auth module")
  ↪ Subagent 1 (explore: "find all files importing auth.py")
      → returns file list
  ↪ Subagent 2 (shell: "run the test suite")
      → returns test results
  ↪ Parent continues with both results in context
```

In practice:

- **Claude Code:** subagents with own context windows, tool restrictions, permission modes, model choices
- **OpenAI Agents SDK:** delegation as "handoffs"—tools like `transfer_to_refund_agent`

How the Model Learns to Delegate

Delegation is presented to the model as tool use. The framework turns subagents into tools—just like function calling.

OpenAI Agents SDK: handoffs are tools

```
billing_agent = Agent(name="Billing agent", ...)
refund_agent = Agent(name="Refund agent", ...)

triage_agent = Agent(
    name="Triage agent",
    handoffs=[billing_agent, refund_agent],
)
# The model sees tools named "transfer_to_billing_agent"
# and "transfer_to_refund_agent" with descriptions.
```

The model doesn't need special "delegation training." It reuses tool-calling capability from Milestone 4. The *orchestration behavior*—knowing when to decompose, how to split work, how to synthesize results—is trained/prompted separately.

Milestone 7: Loop Check

Loop check — subagents

1. *How does the model express the call?* The parent agent launches a subagent the same way it'd call a tool—task description in, result out. The framework converts subagents into tool definitions.
2. *Who executes?* A child agent runtime with its own context, tools, and (optionally) a different model.
3. *What guarantees?* Isolation is the new guarantee: the subagent's context, permissions, and failures are scoped. The parent gets a result without its own context being polluted by exploration noise.

Read: Claude Code subagents (`docs.anthropic.com`), OpenAI Agents SDK handoffs (`openai.github.io/openai-agents-python/handoffs/`)

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Why We Need Another Mechanism

At this point you might ask: "Don't we already have enough layers?" Let's be explicit about what each existing mechanism *can't* do:

- **"Just write a better prompt."** System prompts are per-conversation. 15 different workflows = stuff them all into every conversation (wasting context), or manually pick the right one (error-prone), or build a routing system (reinventing skills).
- **"Use a tool."** A tool does one thing; a workflow is a *recipe* that coordinates many things. A monolithic tool is a black box.
- **"Use an MCP server."** MCP gives building blocks, not instructions for how to combine them.
- **"Use a subagent."** Instructions live in code, not discoverable, not versioned as files, not shareable across teams.

What Skills Actually Are

A skill fills the gap: **reusable, discoverable, file-based workflow expertise that the agent loads on demand.**

	Tool	Skill
What it is	A callable function	A package of instructions + resources
Analogy	A screwdriver	A carpentry manual
Example	`get_weather(city)`	"How to write a quarterly business review"
Invocation	Model calls it	Model *loads and follows* it
Shareable	Via MCP / API	Via filesystem—copy the directory
Discoverable	`tools/list`	Agent scans skill directories automatically

A skill says: "here is the procedure, the checklist, the templates, and the scripts. Follow these steps and use whatever tools you need along the way." It's the difference between giving someone a hammer and giving them a building plan.

How Skills Work (Anthropic's Model)

A skill is a directory with a required `SKILL.md` file plus optional scripts, templates, and resources. Claude discovers skills dynamically using a three-tier system:

1. **Metadata** (always loaded): YAML frontmatter ($\sim$100 tokens)
2. **Instructions** (loaded when triggered): the core procedure
3. **Resources** (loaded as needed): scripts, templates, examples

Why this matters: skills reduce repeated prompt-stuffing and encode organizational best practices. Portable across Claude.ai, Claude Code, and the API.

Read: Anthropic *Introducing Agent Skills*, *Skills explained*, Agent Skills specification

Milestone 8: Loop Check & The Modern Stack

Loop check — skills

1. *How does the model express the call?* It doesn't "call" a skill—it *loads and follows* one. The agent discovers relevant skills and incorporates their instructions.
2. *Who executes?* The agent itself, guided by the skill's instructions (which may use tools, MCP servers, or subagents).
3. *What guarantees?* Consistency, not determinism. Same skill → same *process* every time, even though individual LLM outputs vary. Deterministic procedure, nondeterministic execution.

The modern stack

```
Skills        → reusable workflow expertise
  +
MCP servers   → standardized access to external systems
  +
Subagents     → role specialization and context isolation
  +
A2A           → cross-organization agent collaboration
```

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Real Execution vs. Pretend Execution

Three things look similar in a chat log but are fundamentally different:

A) Real tool execution

The system actually runs code/tools and returns real results. You can see artifacts (files, logs, errors).

Code Interpreter, MCP servers, shell tools.

B) Code generation

The model writes code that *would* call an API. If your runtime executes it, great. If not, it's just a plan.

The PAL lineage.

C) Pure simulation

The model emits text that *looks like* a tool call or terminal output, but nothing actually ran.

Common in the XML-tag era.

In your assignments

Always log: (1) tool request from model, (2) execution trace, (3) result returned, (4) final answer. This makes the difference between A, B, and C unmistakable.

This is P^3 **Proof** infrastructure—without observable execution traces, you can't verify your system actually does what it claims.

The Full Comparison Matrix

Approach	Who decides?	Who executes?	Format	Guarantees	Reusability	Best for
Prompt + format	LLM via prompt	Nobody (text)	Free text/JSON	None (hope)	None	Simple struct. output
Code gen (PAL)	LLM writes code	Interpreter	Code	Syntax only	Low	Computation
ReAct prompting	LLM via prompt	Your glue code	Text conventions	None (hope)	Low	Prototyping
Function calling	LLM via API	Your code	JSON Schema	Valid JSON	Medium	App tools (1–10)
Structured outputs	LLM via API	Your code	Strict JSON	Schema-det.	Medium	Schema compliance
MCP tool	LLM via API	MCP server	JSON-RPC	Schema-det.+disc.	High	Cross-app ecosystems
Subagent	Parent agent	Child runtime	Task delegation	Isolation	High	Complex multi-step
Skill	Agent loads	Agent follows	MD + resources	Consistent process	Very high	Repeatable workflows
A2A	Agent disc. peers	Remote agent	Protocol msgs	Protocol-det.	Very high	Cross-org collab.

Decision Flowchart: "What Should I Use?" (Part 1)

```
START: "I want the model to do X"
  |
  +-- Can the model do X with just a better prompt?
  |   YES → Write a better prompt (Unit 4). Done.
  |   NO  ↓
  |
  +-- Does X require external data or side effects?
  |   NO  → Use structured outputs for reliable format. Done.
  |   YES ↓
  |
  +-- Is X a single function in your own codebase?
  |   YES → Use function calling (+ structured outputs). Done.
  |   NO  ↓
  |
  +-- Do multiple apps/hosts need to call X?
      NO  → Function calling is still fine. Done.
      YES ↓ ... (continued)
```

Decision Flowchart: "What Should I Use?" (Part 2)

```
... (continued from Part 1)
|
+-- Build an MCP server for X.
|   |
|   +-- Does the task require multi-step planning?
|       NO  → Single tool call via MCP. Done.
|       YES ↓
|
+-- Does the subtask need isolation (context, perms, model)?
|   YES → Use a subagent with restricted tools. Done.
|   NO  → Agent loop with MCP tools. Done.
|
+-- Is this a repeatable workflow you want standardized?
    YES → Package it as a skill. Done.
    NO  → You're probably fine with the above.
```

The meta-principle

Start simple (prompts) → add structure when reliability demands it (structured outputs) → add execution when the world demands it (tools) → add standardization when scale demands it (MCP) → add delegation when complexity demands it (subagents) → add expertise when consistency demands it (skills).

In P^3 terms, this flowchart is the **Production** phase: you choose architecture based on where your **Proof** revealed gaps.

Timeline Summary

Year	Milestone	Key idea
pre-2021	Semantic parsing, task-oriented dialogue	NL to structured commands
2021	WebGPT	Train models to use browser as a tool
2022	MRKL, PAL, SayCan, ReAct	Prompting patterns for tool use
2023	Toolformer, Gorilla	Training tool use into model weights
Mar 2023	OpenAI Plugins	Tools described via OpenAPI spec
Jun 2023	Function calling	Structured API channel for tool calls
Nov 2023	JSON mode	Valid JSON output (no schema guarantee)
Aug 2024	Structured Outputs	Schema-enforced via constrained decoding
Nov 2024	MCP announced	Vendor-agnostic tool/resource protocol
Apr 2025	A2A announced	Agent-to-agent interoperability protocol
Oct 2025	Agent Skills	Reusable expertise bundles
2025–26	MCP in production	OpenAI, Anthropic, Cursor operationalize MCP

Security Note: Tools Create New Attack Surfaces

As systems gain tool access (web, files, databases, MCP servers), they inherit new risks. MCP's specification explicitly calls out:

Key security considerations

- **User consent and control:** humans must approve tool access
- **Data privacy:** tools should not leak data across contexts
- **Tool safety:** destructive actions require explicit authorization
- **Prompt injection:** adversarial inputs can manipulate tool calls

This will come up again in Unit 13 (Observability & Debugging), but keep it in mind as you build: **every tool you add is an attack surface.**

In P^3 terms, these are **Production** durability requirements—your Promise must hold under adversarial conditions.

Key Concepts Introduced

- **Tool-use loop** — the key mental model
- **Function calling** — structured API channel
- **Constrained decoding** — schema-enforced output
- **Determinism ladder** — free text → structured outputs
- **MCP** — vendor-agnostic tool/resource protocol
- **Tools vs. resources** — "what can I do?" vs. "what can I see?"
- **Subagents** — delegation with context isolation
- **Skills** — reusable workflow expertise bundles
- **ReAct pattern** — reasoning + acting, interleaved
- **Real vs. pretend execution** — verifiable vs. simulated

Outline

1. Warm-up and Mental Models
2. Milestone 1: Research Roots (pre-2021)
3. Milestone 2: LLMs Learn to Reach for Tools (2021–2022)
4. Milestone 3: Training Models to Use Tools (2023)
5. Milestone 4: Tool Use Becomes a Platform Feature (2023–2024)
6. Milestone 5: MCP (Nov 2024)
7. Milestone 6: Agent-to-Agent Communication — A2A (2025)
8. Milestone 7: Subagents — Delegation Inside a System (2025)
9. Milestone 8: Skills — Reusable Expertise Bundles (Oct 2025)

Foundational Research Papers

PDFs for all of these are in `unit05_readings.zip`.

1. Su et al., *Building Natural Language Interfaces to Web APIs* (2017)
2. Gupta et al., *Semantic Parsing for Task-Oriented Dialog* (2018)
3. Nakano et al., *WebGPT* (2021) — `arxiv.org/abs/2112.09332`
4. Karpas et al., *MRKL Systems* (2022) — `arxiv.org/abs/2205.00445`
5. Gao et al., *PAL: Program-Aided Language Models* (2022) — `arxiv.org/abs/2211.10435`
6. Ahn et al., *SayCan* (2022) — `arxiv.org/abs/2204.01691`
7. Yao et al., *ReAct* (2022) — `arxiv.org/abs/2210.03629`
8. Schick et al., *Toolformer* (2023) — `arxiv.org/abs/2302.04761`
9. Patil et al., *Gorilla* (2023) — `arxiv.org/abs/2305.15334`
10. Li et al., *API-Bank* (2023) — `arxiv.org/abs/2304.08244`
11. Qin et al., *ToolLLM / ToolBench* (2023) — `arxiv.org/abs/2307.16789`
12. Yang et al., *SWE-agent* (2024) — `arxiv.org/abs/2405.15793`

Platform Docs & Protocol Specs

13. OpenAI Function Calling guide — `platform.openai.com/docs/guides/function-calling`
14. OpenAI Structured Outputs — `platform.openai.com/docs/guides/structured-outputs`
15. Anthropic Tool Use docs — `docs.anthropic.com/en/docs/build-with-claude/tool-use/overview`
16. MCP announcement — `anthropic.com/news/model-context-protocol`
17. MCP specification — `modelcontextprotocol.io/specification/latest`
18. A2A announcement — `developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability`
19. A2A specification — `google.github.io/A2A/specification/`
20. Anthropic Agent Skills overview — `anthropic.com/news/skills`
21. Skills explained (comparison) — `claude.com/blog/skills-explained`
22. The Complete Guide to Building Skills for Claude (PDF)

Context & background:

- *Effective context engineering for AI agents* (Anthropic, Sep 2025)—bridges prompt engineering to the agentic context management that tool-using systems require.

Thank you!